

CS3404: GUI Programming

Mini Project Report

Product Explorer

A Data-Driven E-Commerce Single Page Application built with Vue 3, TypeScript, Tailwind CSS and DummyJSON

Name: CROOS W.G.C.M

Registration Number: EG/2023/5540

Degree: Computer Engineering

Repository: <https://github.com/gyrika/GUI-Project>

Submission Date: 21 September 2026

1. Project Overview

Product Explorer is a Single Page Application (SPA) that lets users browse, search and filter a product catalogue, view full product details, manage a persistent shopping cart and switch between light and dark themes. All product data is consumed live from the public DummyJSON REST API.

The project was built to meet the CS3404 mini project requirements: a strictly typed front-end architecture, asynchronous API consumption, clean component and state management, responsive utility-first styling and a meaningful Git history. In addition to the mandatory requirements, three of the four bonus features were implemented: a persistent shopping cart, dynamic routing and dark mode.

1.1 Technology Stack

Layer	Technology	Purpose in this project
Framework	Vue 3	Composition API with <code><script setup lang="ts"></code> in every component
Language	TypeScript	Strict mode enabled; typed props, emits, stores and API responses
Build tool	Vite	Dev server and production bundling; <code>@</code> path alias to <code>/src</code>
Styling	Tailwind CSS v4	Utility-first styling via the <code>@tailwindcss/vite</code> plugin, class-based dark mode
Routing	Vue Router	History-mode routing with named and dynamic routes
State	Pinia	Global, persistent shopping cart store
Runtime / package manager	Bun	Dependency installation and script execution
Type checking	vue-tsc	Full type check of <code>.vue</code> and <code>.ts</code> files before every production build
Data source	DummyJSON API	<code>/products</code> , <code>/products/{id}</code> , <code>/products/category-list</code>

2. Features Implemented

2.1 Core Requirements

Requirement	Implementation
Data fetching	Products are loaded from <code>/products</code> through a typed service layer (<code>services/api.ts</code>) and exposed to the UI by the <code>useProducts()</code> composable, which tracks products, loading and error state. Categories are loaded separately from <code>/products/category-list</code> to populate the filter.
Search	Live, case-insensitive search by product title. The query is trimmed and

Requirement	Implementation
	normalised, and results update instantly through a computed property as the user types.
Filtering	A category dropdown filters products by category. Search and category filters are combined (a product must match both), and an "All" option shows every category.
Detail view	Clicking a product card navigates to a dedicated route, /product/:id, which fetches that product and shows its image, category, title, description, price, rating, stock and brand.
Loading & error states	Animated skeleton cards are shown while data loads. Failed requests show an error panel with a "Try again" button. When no products match the filters, an empty state with a "Reset filters" button is shown.
Responsive layout	Tailwind grid and flex utilities adapt every page to mobile, tablet and desktop (see Section 5.1).
Component architecture	The UI is split into 3 routed views and 6 reusable components, supported by a service layer, composables and a Pinia store (see Section 3).

2.2 Bonus Features

Shopping cart with persistence (Pinia). A global Pinia store (stores/cart.ts) manages cart items. Users can add a product from its detail page, increase or decrease quantities, remove individual items and clear the cart. Decreasing an item with quantity 1 removes it. The store exposes computed totals (item count and price), which the navigation bar shows live on every page. The cart is saved to localStorage whenever it changes and restored on reload, so it persists across page refreshes. A dedicated /cart page shows line totals, a sticky order summary and an empty-cart state.

Dynamic routing (Vue Router). The application uses three named routes: home (/), product-detail (/product/:id) and cart (/cart). Product cards link using the named route and its id parameter, so every product has a unique, shareable URL, and the browser back and forward buttons work as expected.

Dark mode. A theme toggle in the navigation bar switches between light and dark themes using Tailwind's dark: modifier. On first visit the app follows the operating system preference (prefers-color-scheme). After that, the user's choice is saved to localStorage. The theme is applied in main.ts before the app mounts, which avoids a flash of the wrong theme on page load.

3. Architecture

The application follows a layered architecture that separates concerns into four layers: types, data access, state and UI. Each layer depends only on the layers below it, which keeps components small and focused on presentation.

3.1 Component Hierarchy

Figure 1 shows how components are nested at runtime. App.vue is a thin shell that renders the NavBar on every page and a RouterView that swaps between the three page-level views. Only HomeView composes further child components; the detail and cart views render their own content directly.

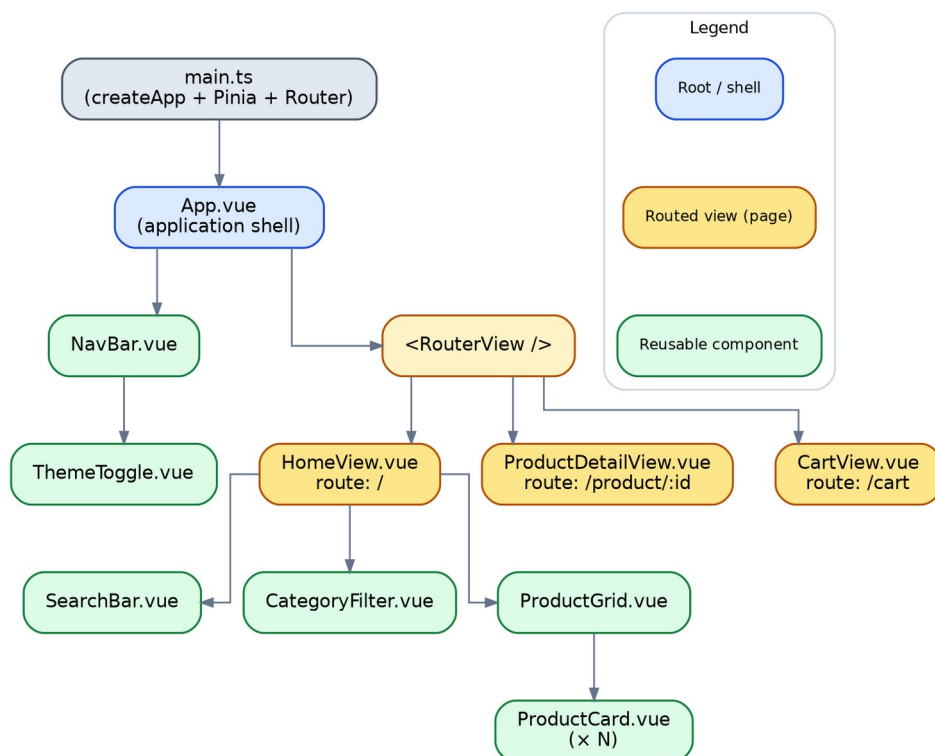


Figure 1: Component hierarchy

3.2 Project Structure

```
src/
├── main.ts                # App bootstrap: Pinia, Router, theme init
├── App.vue                # Application shell (NavBar + RouterView)
├── style.css              # Tailwind import, dark-mode variant, base styles
├── types/product.ts       # TypeScript interfaces for API data
├── services/api.ts        # Typed DummyJSON API functions
├── composables/
│   ├── useProducts.ts     # Product list + loading/error state
│   └── useTheme.ts        # Shared light/dark theme state
├── stores/cart.ts         # Pinia cart store with persistence
├── router/index.ts        # Route definitions
├── views/                 # Routed pages
│   ├── HomeView.vue
│   ├── ProductDetailView.vue
│   └── CartView.vue
└── components/           # Reusable UI components
```

- NavBar.vue
- ThemeToggle.vue
- SearchBar.vue
- CategoryFilter.vue
- ProductGrid.vue
- ProductCard.vue

3.3 Component Responsibilities

Component	Props / Emits	Responsibility
App.vue	–	Global layout, background and max-width container; hosts NavBar and RouterView.
NavBar.vue	– (reads cart store)	Brand link, navigation, live cart count and total, theme toggle. Stacks vertically on small screens.
ThemeToggle.vue	– (uses useTheme)	Accessible button that toggles the theme (aria-pressed, dynamic aria-label).
SearchBar.vue	modelValue / update:modelValue	Controlled search input, used with v-model.
CategoryFilter.vue	categories, modelValue / update:modelValue	Controlled category dropdown, used with v-model.
ProductGrid.vue	products: Product[]	Responsive 1/2/3-column grid that renders a ProductCard for each item.
ProductCard.vue	product: Product	Thumbnail, category, title and price; links to the detail route; hover lift and zoom effects.
HomeView.vue	–	Owens search and filter state, computes filtered products, handles loading, error and empty states.
ProductDetailView.vue	– (route param id)	Validates the id, fetches one product, renders details and the Add to Cart action.
CartView.vue	– (reads cart store)	Cart line items, quantity controls, order summary and empty-cart state.

3.4 Data Flow and State Management

Figure 2 shows how data moves through the layers. State is deliberately placed at the lowest scope that needs it:

- **Local component state:** the search query and selected category live inside HomeView, because no other page needs them. They are passed down to SearchBar and CategoryFilter as props and updated through emitted update:modelValue events (the v-model pattern), so the child components stay stateless and reusable.
- **Composable (per-use state):** useProducts() encapsulates the fetch, loading and error logic for the product list, keeping it out of the view and making it reusable.

- **Shared composable (module-level state):** useTheme() keeps one theme ref at module level, so every component that calls it shares the same theme without needing a store.
- **Global Pinia store:** the cart must be read by NavBar, ProductDetailView and CartView at the same time and must survive reloads, so it lives in a Pinia setup store. A deep watcher saves the cart to localStorage after every change.

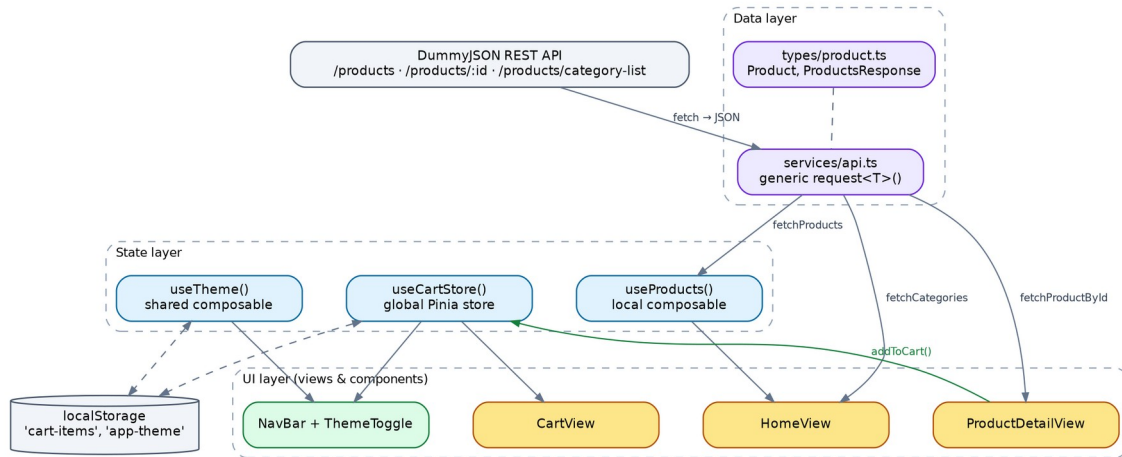


Figure 2: Layered architecture and data flow

3.5 Routing

Path	Name	View
/	home	HomeView: product catalogue with search and filter
/product/:id	product-detail	ProductDetailView: single product details and Add to Cart
/cart	cart	CartView: cart contents and order summary

ProductDetailView watches the computed product id with { immediate: true } instead of using onMounted. Vue Router reuses the same component instance when only the :id parameter changes, so a watcher guarantees the correct product is always fetched.

4. Type Safety

TypeScript runs in strict mode and the codebase contains no any types. The build script runs `vue-tsc --noEmit` before `vite build`, so a type error stops the production build.

4.1 API Interfaces

Every DummyJSON response has a matching interface in `types/product.ts`, including nested objects:

```
export interface Product {
  id: number; title: string; description: string; category: string
  price: number; discountPercentage: number; rating: number; stock: number
  tags: string[]; brand: string; sku: string; weight: number
  dimensions: ProductDimensions // { width, height, depth }
```

```

reviews: ProductReview[]           // { rating, comment, date, ... }
meta: ProductMeta                  // { createdAt, updatedAt, barcode, qrCode }
images: string[]; thumbnail: string
// ...plus warranty, shipping and availability fields
}

export interface ProductsResponse {
  products: Product[]; total: number; skip: number; limit: number
}

```

4.2 Typing Techniques Used

- **Generic request function:** a single `request<T>(path)` helper handles fetch and HTTP errors. Each API function declares its exact return type, such as `Promise<ProductsResponse>`.
- **Derived types:** `CartItem` uses indexed access types (`Product['price']`) and `addToCart()` accepts `Pick<Product, 'id' | 'title' | 'price' | 'thumbnail'>`. If the `Product` interface changes, the cart types stay in sync automatically.
- **Runtime type guards:** data read from `localStorage` cannot be trusted, so it is parsed as unknown and validated with the `isCartItem()` and `isTheme()` type guards before use. Corrupted or edited storage never crashes the app.
- **Union types:** type `Theme = 'light' | 'dark'` prevents invalid theme values.
- **Typed component contracts:** props use `defineProps<{ ... }>()` and events use the typed tuple syntax of `defineEmits`. DOM event targets are narrowed with `instanceof HTMLInputElement` rather than type-cast.
- **Safe error handling:** caught errors are typed as unknown and narrowed with `instanceof Error` before their message is read.
- **Route parameter validation:** the `:id` parameter (typed as `string | string[]`) is normalised and accepted only if it is a positive integer. Otherwise an "Invalid product id" error is shown instead of sending a bad request.

5. UI / UX Design

5.1 Responsive Behaviour

Element	Mobile (< 640px)	Tablet (sm / md)	Desktop (lg / xl)
Product grid	1 column	2 columns	3 columns (xl)
Search + filter panel	Stacked	Side by side (md)	Side by side
Navigation bar	Stacked, full-width links	Links in a row	Brand and links on one row (lg)
Product detail	Image above details	Image above details	Image and details side by side
Cart page	Single column	Row-style cart items	Items + sticky summary sidebar

5.2 Visual Design System

- A consistent slate neutral palette is used for text, borders and surfaces, with emerald as the accent for cart elements and red for destructive actions and errors.
- Shapes are consistent: rounded-2xl cards and inputs, rounded-3xl panels and rounded-full buttons.
- Hierarchy is reinforced with small uppercase tracked labels above large semibold headings.
- The navigation bar uses a translucent background with backdrop blur, and the page has a soft radial gradient background in both themes.

5.3 Interaction and Feedback

- Product cards lift slightly on hover, and their images zoom subtly. All buttons and links have hover transitions.
- Skeleton loaders that match the final layout reduce perceived loading time. Theme changes animate smoothly through colour transitions.
- Every state is handled explicitly: loading, error with retry, no search results with reset, empty cart with a link back to shopping, and a note when an item is already in the cart.

5.4 Accessibility

- Form controls are wrapped in `<label>` elements. Icon-style buttons (+ and -) have descriptive `aria-label` attributes.
- The theme toggle exposes its state with `aria-pressed`. Error panels use `role="alert"` so screen readers announce them.
- Visible focus rings are provided, all product images have alt text, and images below the fold load lazily.
- Colour contrast is maintained in both light and dark themes.

6. Version Control

The project was developed incrementally with Git and published on GitHub. Each commit contains a single, self-contained step, and its message describes what changed. The history follows the order in which the architecture was built: types first, then the service layer, state, UI, routing and finally the bonus features. This means every commit in the history leaves the application in a working state.

Date (2026)	Commit message
24 Mar	Initialize Bun-based Vue 3 TypeScript project with Tailwind
24 Mar	Add strict TypeScript interfaces for product data
24 Mar	Create reusable DummyJSON API service functions
24 Mar	Add reusable products composable with loading and error state
24 Mar	Render products in responsive grid with loading and error states
25 Mar	Add responsive navbar and clean application shell

Date (2026)	Commit message
25 Mar	Implement reactive product search by title
25 Mar	Add category filter integrated with product search
25 Mar	Configure Vue Router with home and product detail routes
25 Mar	Implement dynamic product detail page
29 Mar	Add typed Pinia cart store and add-to-cart flow
29 Mar	Persist cart state with localStorage
29 Mar	Create cart page with totals and empty state
29 Mar	Add persistent dark mode toggle across the app
29 Mar	Polish UI and clean up project for submission

7. Use of Generative AI

AI assistance was used during development in line with the course GenAI policy. The full log of aims and the exact prompts used is provided in `prompts.txt`, which is included in the submission ZIP file.

8. Challenges and Solutions

Type narrowing inside callbacks. `vue-tsc` reported that `product.value` was "possibly null" inside the `.some()` callback of the `isInCart` computed property, even after a null check.

TypeScript does not keep a narrowed ref property inside a nested function. This was fixed by copying the value into a local constant before the check, which TypeScript can narrow safely.

Untrusted persisted data. `localStorage` can contain stale or edited data. Instead of casting the parsed JSON directly to `CartItem[]`, the data is treated as unknown and filtered through a type guard. Invalid entries are discarded silently.

Component reuse between routes. Navigating from one product to another reuses the same `ProductDetailView` instance, so `onMounted` would not fire again. Watching the computed route id with `immediate: true` ensures the data always matches the URL.

Class-based dark mode in Tailwind v4. Tailwind v4 bases dark: on the OS media query by default. A `@custom-variant dark` rule in `style.css` switches it to a `.dark` class on the `<html>` element, so the user's own choice can override the OS setting.

9. Future Improvements

- **Pagination:** `DummyJSON` returns 30 products per request by default. Adding pagination or infinite scroll with the `limit` and `skip` parameters would expose the full catalogue.
- **Authentication simulation:** log in through `/auth/login`, store the returned JWT in `localStorage` and show a Log In / Log Out state in the navigation bar.

- **Server-side search:** use the /products/search endpoint with a debounced input to search the whole catalogue rather than only the loaded page.
- **Richer detail page:** an image gallery using all product images, customer reviews, and discount pricing.
- **Testing:** unit tests for the cart store and composables with Vitest.

10. Running the Project

Install dependencies and start the development server with Bun:

```
bun install
bun run dev      # development server
bun run build    # type-check + production build
```

The project can also be run with npm:

```
npm install
npm run dev
```